STUDENT RECORD MANAGEMENT SYSTEM

1.Introduction

The **Student Record Management System** is a console-based application developed using **Python**.
The main objective of this project is to manage student information efficiently using **file-based storage** instead of a database.

The system allows users to:

- Add student records
- View stored records
- Search for students
- Update student details
- Delete student records

This project demonstrates the use of **core programming concepts**, **file handling**, **data structures**, and **clean code practices**.

2. Project Objectives

- To design a simple student management system
- To store and manage student records using CSV files
- To implement CRUD operations
- To practice file handling and structured programming
- To ensure data persistence without using any database
- To build a menu-driven console application

3. Scope of the Project

Included

- Student record creation
- Record searching
- Record modification
- Record deletion
- Persistent storage using CSV files
- Input validation and error handling

4. Technology Stack

Component	Description
Programming Language	Python 3
Data Storage	CSV File
Libraries Used	csv, os
Platform	Cross-platform (Windows/Linux/macOS)

5. System Requirements

Hardware Requirements

- Any system capable of running Python
- Minimum 2GB RAM

Software Requirements

- Python 3.10.1
- Command Line Interface (Terminal / Command Prompt)

6. Data Model

Each student record contains the following fields:

Field Name	Description
Student_ID	Unique identifier for each student
Name	Student name
Age	Student age
Course	Enrolled course
Marks	Academic marks

7. Functional Requirements

7.1 Add Student

- User inputs student details
- Student ID must be unique
- Data is stored in CSV file

7.2 View All Students

- Displays all stored student records

- Output is formatted for readability

7.3 Search Student

- Search using Student ID
- Displays full student details if found

7.4 Update Student

- Modify existing student information using Student ID
- Allows partial updates

7.5 Delete Student

- Remove a student record permanently using Student ID

7.6 Save & Load Data

- Data is automatically loaded on program start
- Data is saved immediately after any modification

8. Non-Functional Requirements

- Code must be readable and well-commented
- No database usage
- Fast execution for small to medium datasets
- Reliable file handling
- Proper error handling

9. Program Flow

1. Program starts
2. CSV file is checked and initialized
3. Main menu is displayed
4. User selects an operation
5. Selected operation is executed
6. Data is saved to CSV
7. Menu repeats until user exits

10. Error Handling

- Handles missing or corrupted CSV file
- Prevents duplicate Student IDs
- Displays user-friendly error messages
- Prevents crashes due to file read/write errors

11. Assumptions

- Student ID is unique
- Users provide valid input for age and marks
- System is used by a single user at a time
- CSV file size is manageable

12. Limitations

- No GUI interface
- No role-based access
- No advanced search or sorting filters
- No encryption or security features

13. Conclusion

The Student Record Management System successfully fulfills all project requirements using file-based data storage.

It demonstrates strong understanding of:

- Python programming
- File handling
- Modular design
- Data validation

This project is suitable for academic evaluation, interviews, and beginner-level software demonstrations.